Architecture

Software Architecture

Software Architecture describes the building blocks of a software systems and their relations as well as the interactions with external elements. The idea is to project the systems along different views to describe it.

QUALITY ATTRIBUTES | STAKEHOLDERS | MACROSCOPIC | ECOSYSTEM | DECISIONS

Principles of software engineering

Over the years, practice and experience lead software engineers to establish some principles and laws:

- The Conway's Law
- SOLID
- YAGNI
- KISS
- Law of Demeter

Conway's law

Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.

— Melvin E. Conway

S.O.L.I.D

- Single-responsibility principle
  - A class should only have a single responsibility, that is, only changes to one part of the software's specification should be able to affect the specification of the class.
- Open–closed principle
  - "Software entities ... should be open for extension, but closed for modification."
- Liskov substitution principle
  - "Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program." Interface segregation principle
  - "Many client-specific interfaces are better than one general-purpose interface."
- Dependency inversion principle
  - One should "depend upon abstractions, [not] concretions."

YAGNI

"You aren't gonna need it" (YAGNI) is a principle of extreme programming (XP) that states a programmer should not add functionality until deemed necessary. XP co-founder Ron Jeffries has written: "Always implement things when you actually need them, never when you just foresee that you need them." Other forms of the phrase include "You aren't going to need it" and "You ain't gonna need it".

KISS

KISS, an acronym for "keep it simple, stupid" or "keep it stupid simple", is a design principle noted by the U.S. Navy in 1960.

The KISS principle states that most systems work best if they are kept simple rather than made complicated. Therefore, simplicity should be a key goal in design, and unnecessary complexity should be avoided.

The law of Demeter

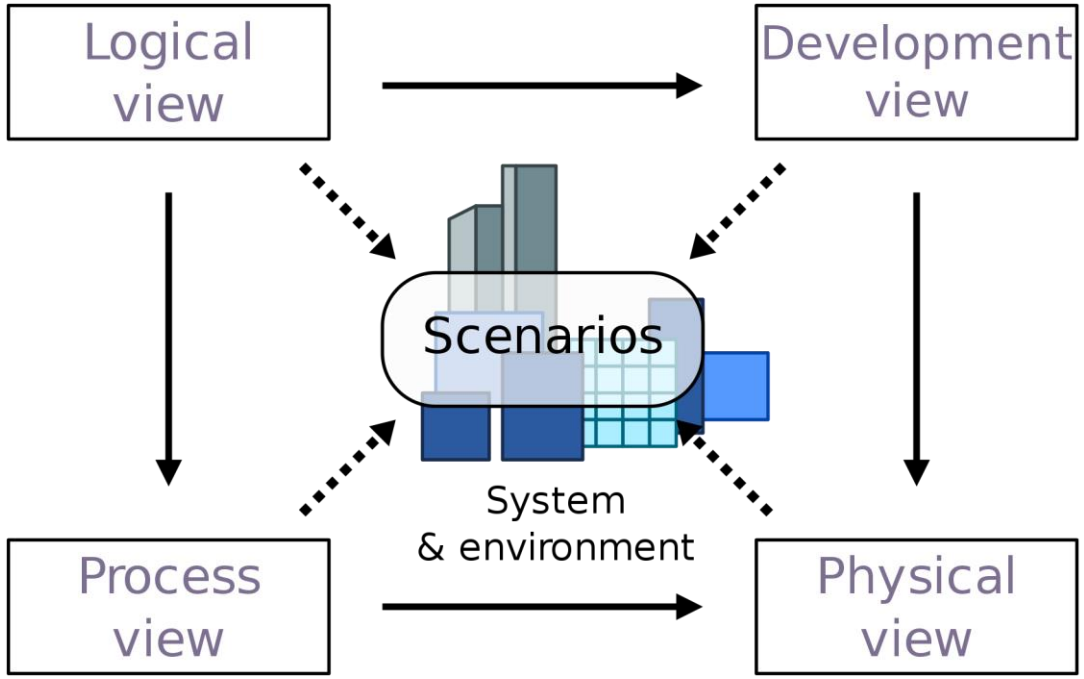
Principle of least knowledge

- Each unit should have only limited knowledge about other units: only units "closely" related to the current unit.
- Each unit should only talk to its friends; don't talk to strangers.
- Only talk to your immediate friends.

While following this principle helps the maintainability but if abused it leads to the creation of many unnecessary wrappers.

Main architecture views

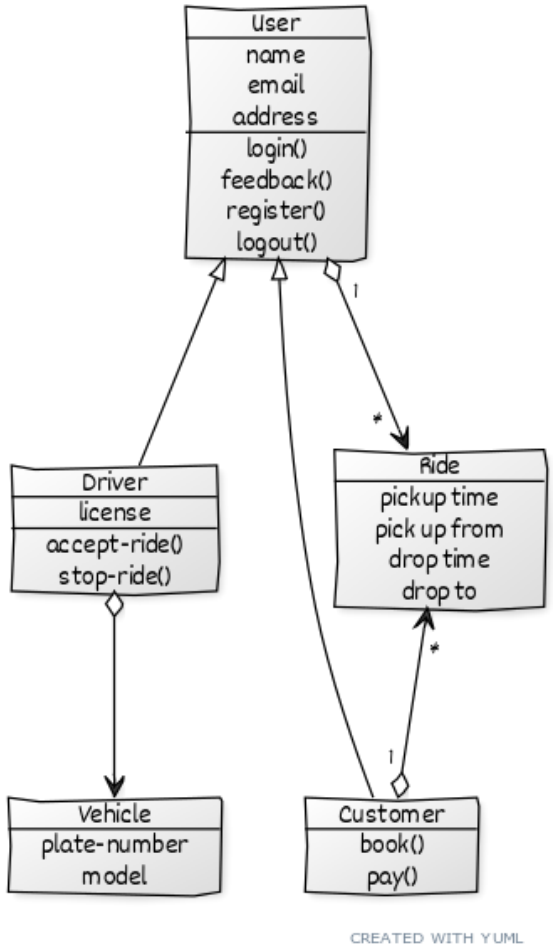
- Logical View: is a static or snapshot view of the system (class diagram, state diagram)
- Process View: is a dynamic view of the system, it describes the runtime behavior (sequence diagram, activity diagram)
- Development View: describes a development team perspective, explains how components interact and are packaged together (component and package diagrams)
- Physical View: how it is deployed physical on the hardware (deployment diagrams)
- Scenarios: gives a high-level understanding of the purpose of the system (use cases)



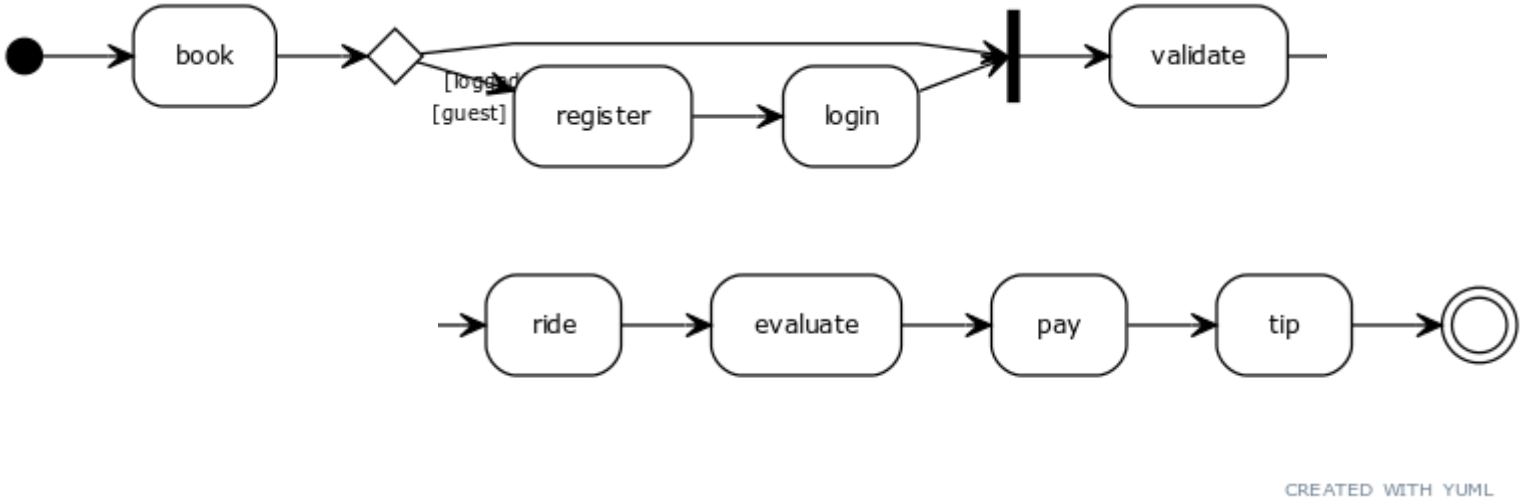
Main architecture views

- Don’t give too much details (otherwise you’ll end up in implementation mode)
- Either use a formal format and stick to it (e.g., UML) or use ad hoc format BUT never use UML without respective the formalism. People will get very confused.

Logical View



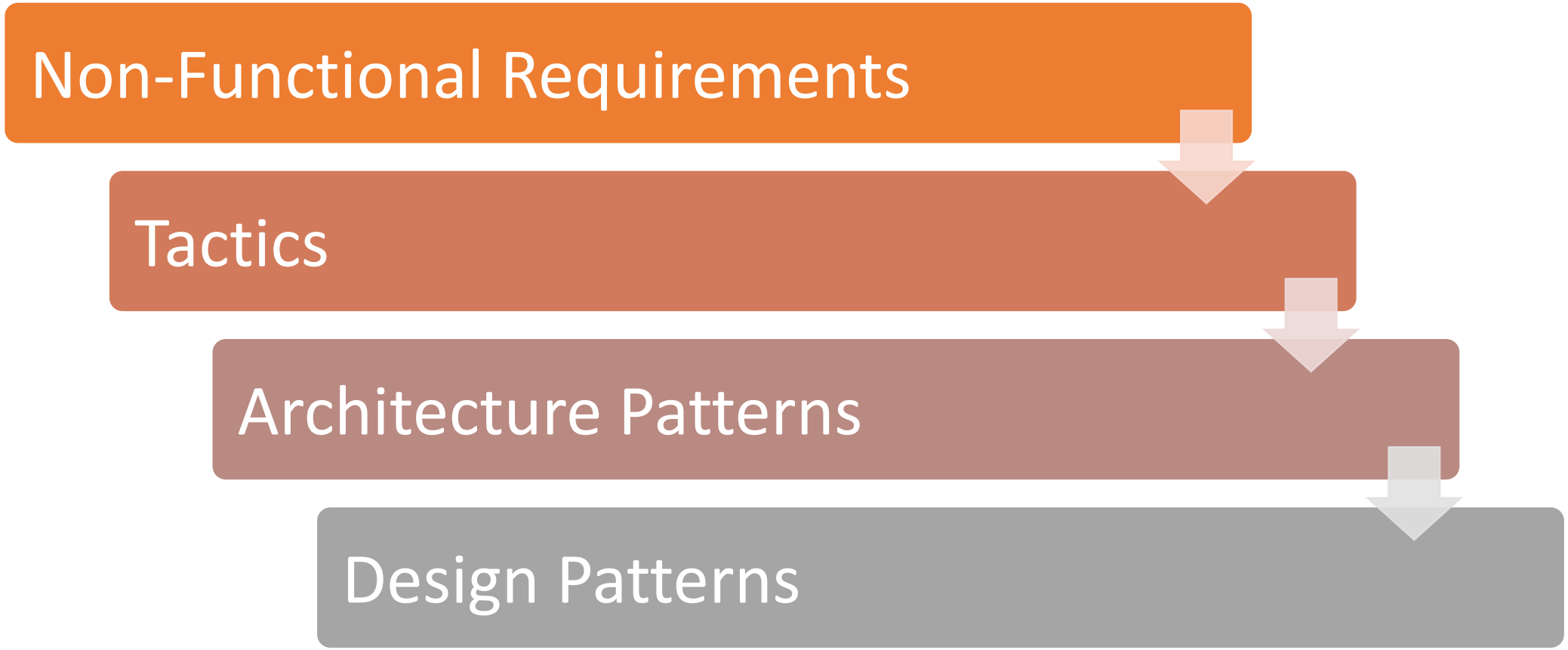
Process View



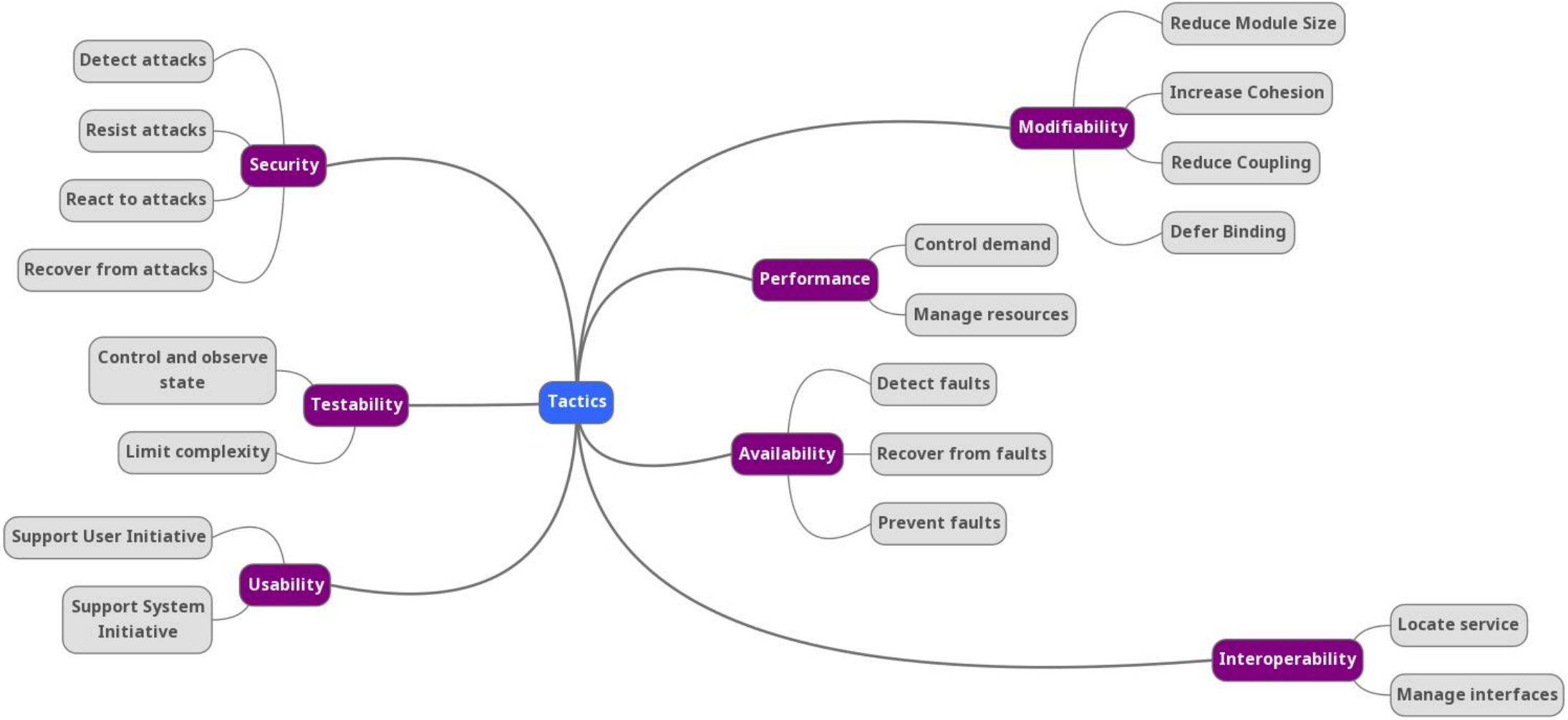
Architecture Views

Establish a logical view and a process view of your project

Software Architecture Process

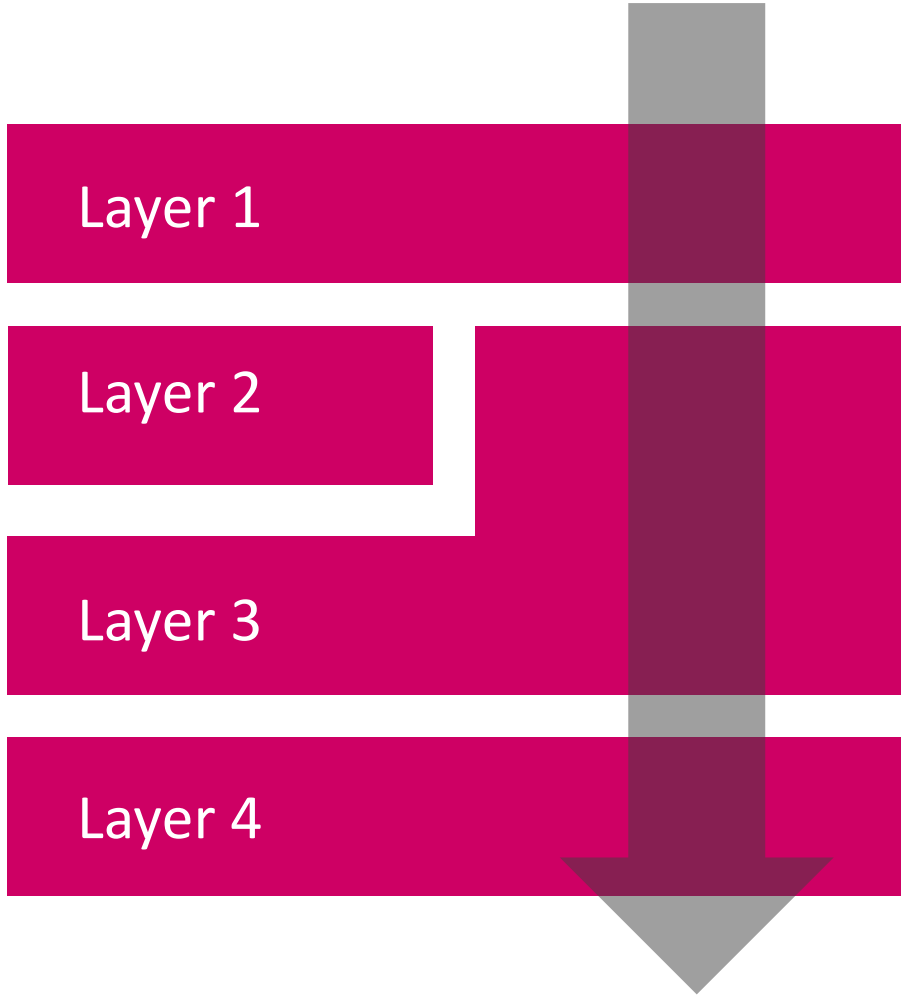


Tactics


Tactics
Security
Detect attacks
Resist attacks
React to attacks
Recover from attacks
Testability
Control and observe state
Limit complexity
Usability
Support User Initiative
Support System Initiative
Modifiability
Reduce Module Size
Increase Cohesion
Reduce Coupling
Defer Binding
Performance
Control demand
Manage resources
Availability
Detect faults
Recover from faults
Prevent faults
Interoperability
Locate service
Manage interfaces

Layered

The layered architecture pattern defines a sequence of layers. Each components is associated to exactly one layer and no lower layer can call a higher layer. The contract between each layer is clearly defined and thus it is easy to replace one layer by another.



Layer n can only call layer n+1

Layered

At least 2 layers

Each component is in exactly one layer

No lower layer can access higher layer

Additional complexity

Negative impact on performance

Reduce coupling

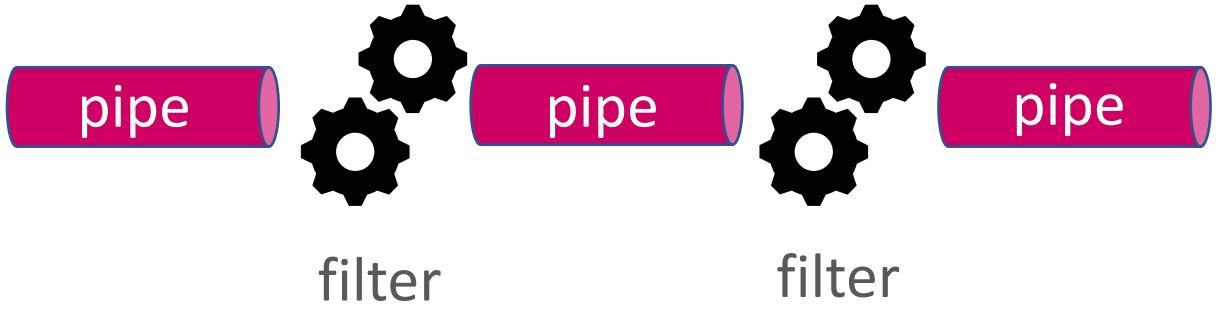
Increase Cohesion

Layered Pattern

Find concrete example of software that implements the layered pattern

Pipe and filters

Define a sequence of transformation of streams of data. Data gets consumed by a filter via it input port, get transformed and then passed through a pipe to the next filter. One filter can consume from and to several pipes.



Pipe and filters

Filters must not have states. They should be idempotent

Might have a negative impact on reliability and consistencies because of additional intermediaries.

Not adapted for interactive systems or long running computations

Reduce coupling

Increase Cohesion

Manage Resources

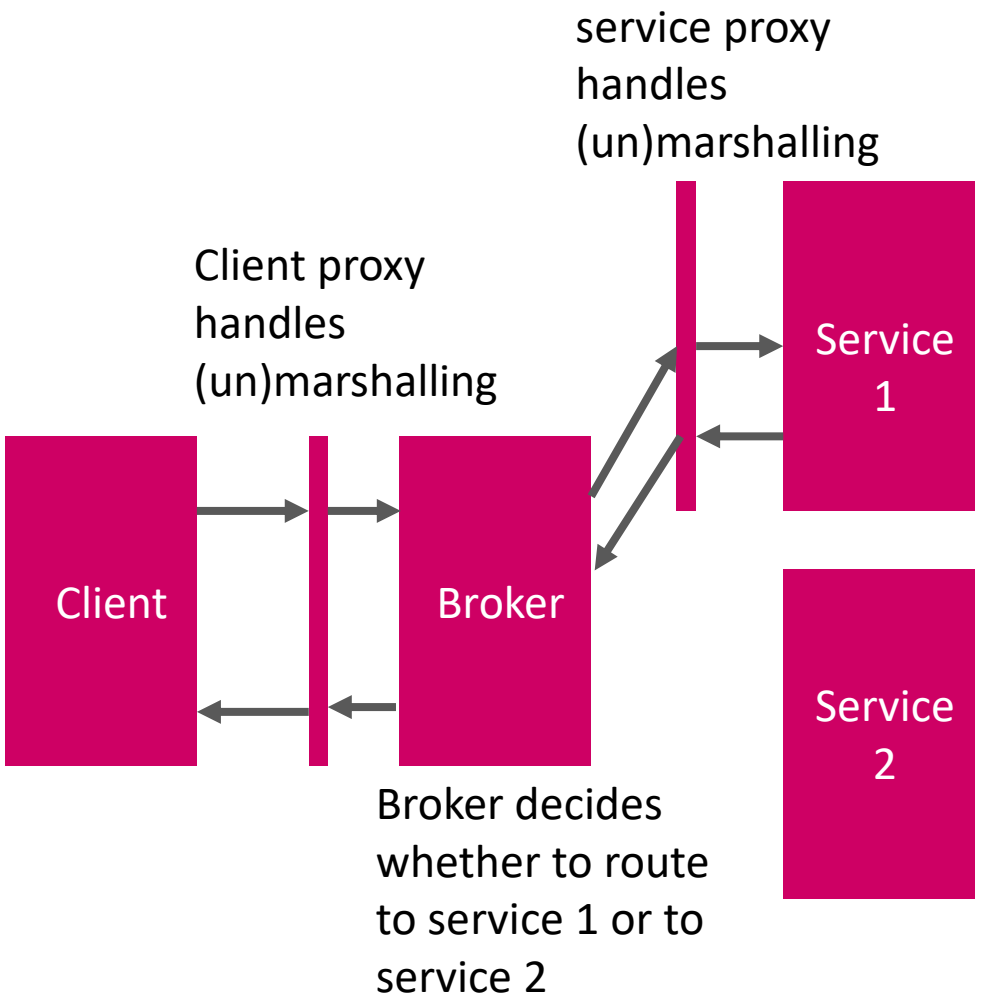
Pipe and Filters Pattern

Find concrete example of software that implements the pipe and filters pattern

Broker

Split the consumers of a service from the providers by inserting a new component called a broker. Consumer does not have any knowledge of the provider

The broker gets the request and then forward it to the provider given internal or external routing conditions. The result is then returned to the broken that in turns transmits it to the consumer.



Broker

Broker often induces a state between client and services

Requires a contract between client and service

Additional complexity

Negative impact on performance

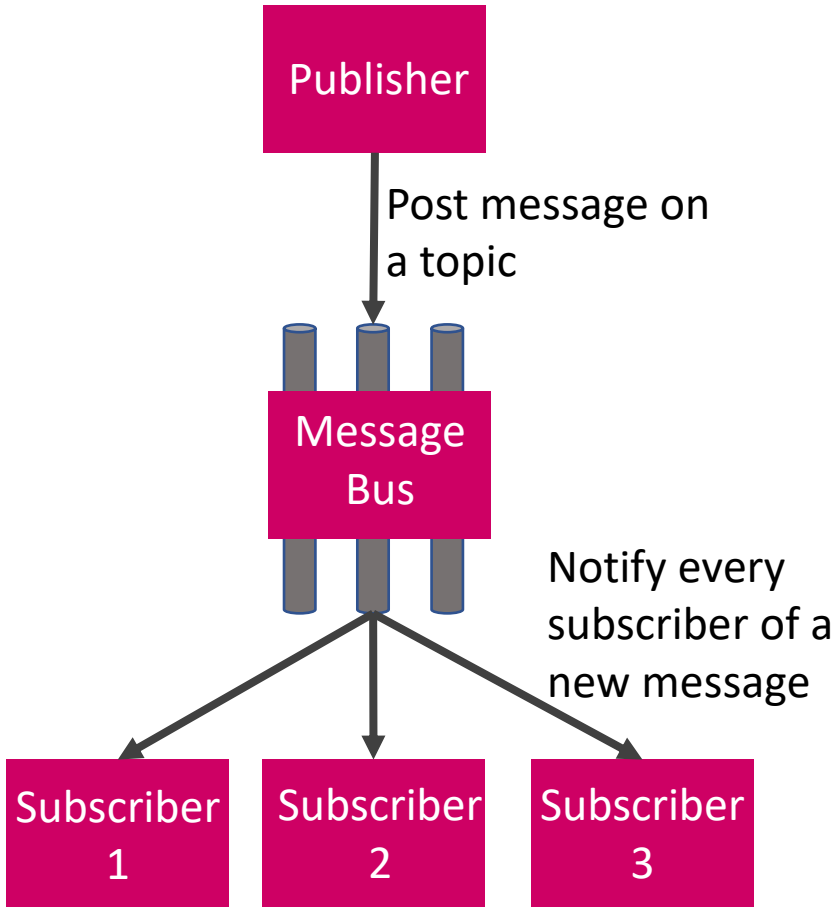
Single point of failure

More difficult to test

Reduce coupling
Defer Binding
Control Demand
Manage resources
Locate service

Publish-Subscribe

Introduce asynchronous messaging to decouple senders and receivers. It ensures that the sender is not blocked until all the consumers received the message and avoid the consumers to have to poll to know whether there is a new message. Furthermore, senders and receivers are unknown of each other.



Publish-Subscribe

Rely on complex technologies

Security policies are mandatory

Subscription handling can become messy

Message order is not guaranteed. Idempotency is required

Repeated message might cause issues

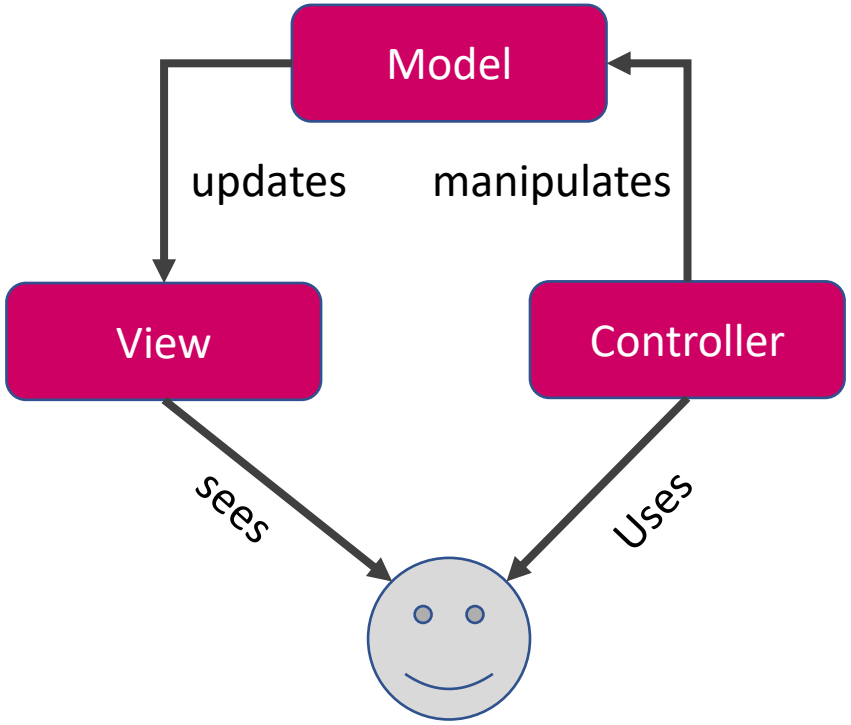
Message expiration

Reduce coupling
Increase cohesion
Defer Binding
Manage interfaces
Control Demand
Manage resources

Model View Controller

(M)odel-(V)iew-(C)ontroller is meant for User Interfaces. It splits the responsibility in 3 components:

1) The model that data and the business logic of the application
2) The view is providing the visual support
3) The controller deals with input and translate them to command to pilot the model and the view.



Model View Controller

Rely on technologies

Responsibilities are sometimes fuzzy

Cascading modification

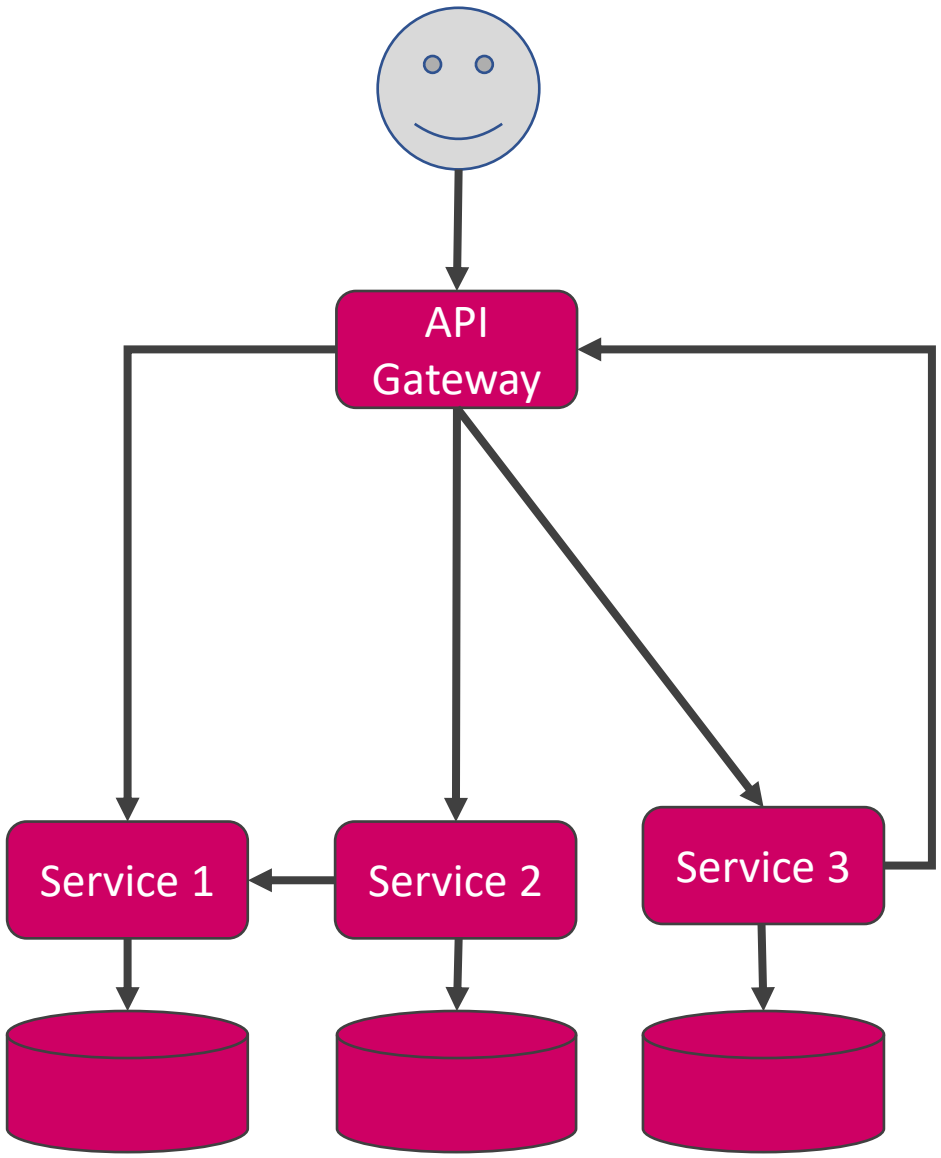
Reduce coupling
Increase cohesion
Defer Bindings

MVC

Find concrete example of framework that help you implement the MVC pattern

Micro-services

Developing a high number of small services over a monolithic application. It enables to be much more granular in the deployment. Each element is easier to test in isolation. Well adapted to small agile teams. Finally, it eliminates the long-term commitment to a particular technology as every component can use a different stack.



Micro-services

Rely on a multitude of technologies

Relies on many other patterns (e.g., broker)

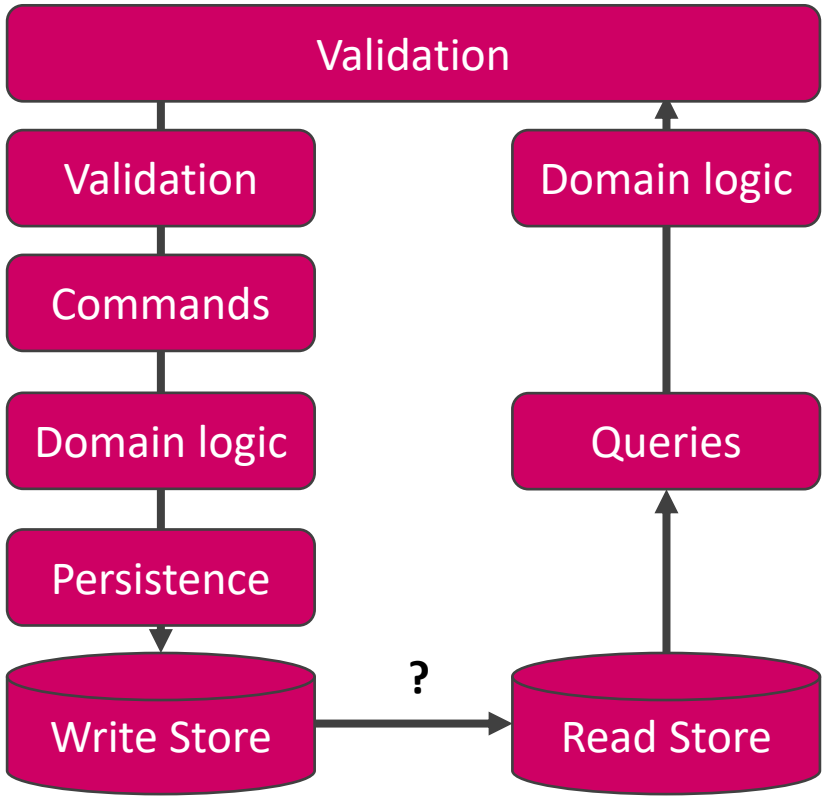
Hype!?

Testing in integration is difficult
Deployment is complex
Increase memory consumption
Potentially very chatty

Reduce module size
Reduce coupling
Increase cohesion
Defer Bindings
Control demand
Locate service
Manage Interfaces

(C)ommand (Q)uery (R)esponsibility (S)egregation

The primarily objective of CQRS is to read and update data from a data store. It recognize that the model for reading and processing (denormalized) is often contradictory with the one optimal for storage (normalized). The other advantage is clear separation from a read and write authorization. This patterns scales usually much better than having the same model for read and write. The pattern is eventually consistent.



(C)ommand (Q)uery (R)esponsibility (S)egregation

Rely on a multitude of technologies

Usually eventual consistency is implemented by publish-subscribe

Additional complexity

Eventual consistency is difficult to control

Reduce coupling

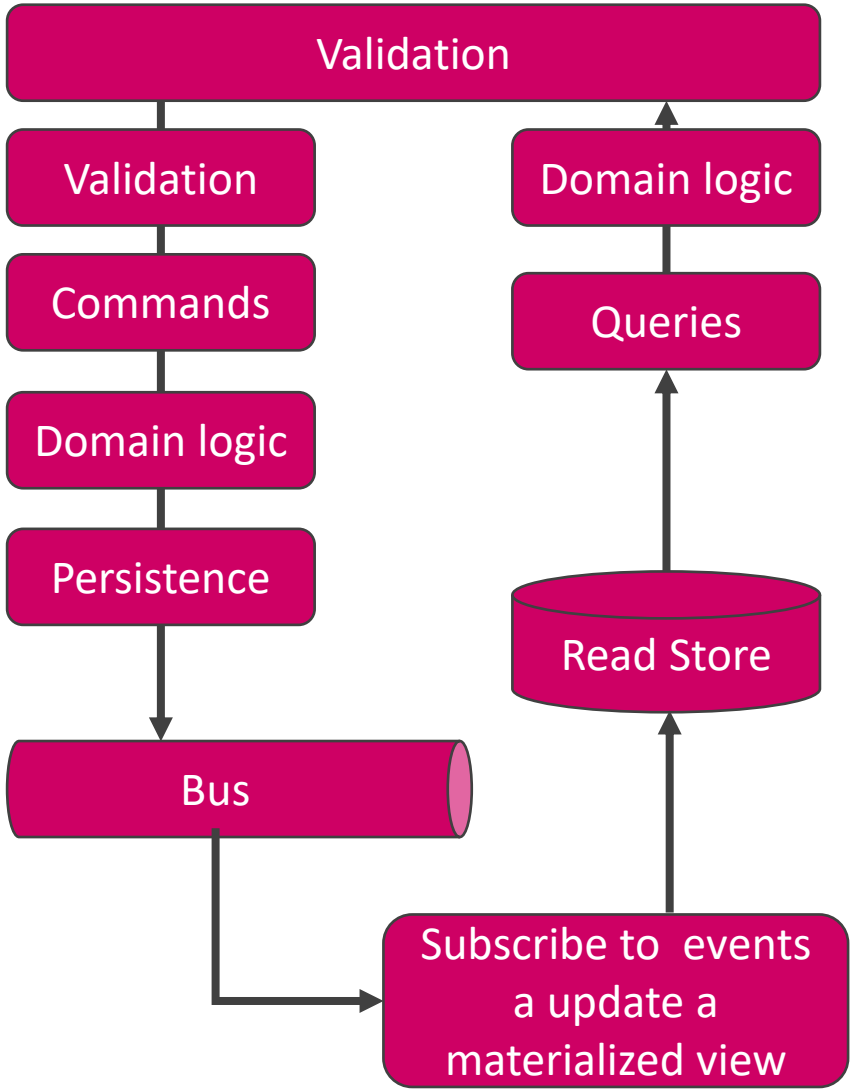
Increase cohesion

Control demand

Manage resources

Event Sourcing

Instead of storing the current state of an object, a sequence of event that lead to that state are stored. As the persistence is append only, it is usually more performant for persistence. Storing small updates is faster than complete objects.
It also provides consistency and full audit, history.
This is especially useful to implement compensation in the case of eventual consistency.
For query performance, a cache of the object state is maintained but it is not the source of truth.



Event Sourcing

Rely on a multitude of technologies

Used in conjunction with CQRS (the write part)

Additional complexity

Eventual consistency is difficult to control

Requires a cache or materialized views of the state of the object

Reduce coupling

Increase cohesion

Control demand

Manage resources

NFR ➔ Tactics ➔ Patterns

Create an architecture that implements Micro-services, MVC, and Layered. What NFRs did drive the decision to use these patterns